Function Approximation and TD-Learning

Consider the infinite-horizon discounted setting with unknown model.

To estimate $Q^\pi(s,a)$ for policy π, i.e.,

$$Q^\pi(s,a) = \mathbb{E}_{\substack{A_t \sim \pi \\ S_{t+1} \sim P(\cdot|S_t,A_t)}} \left[\sum_{t=0}^{\infty} \gamma^t R(S_t, A_t) \mid S_0 = s, A_0 = a \right],$$

using supervised learning, one can use the roll-out process to generate a data set with features and labels according to

$$\begin{cases} x^i = (s^i, a^i) \\ y^i = \sum_{t'=0}^{t} r_{t'} \end{cases}.$$

with geometrically distributed t, γ is not needed for creating an unbiased estimate.

However, this MC-based method defers the update of the estimate to after a trajectory is generated.

* Using TD-based methods, we can update the estimate online.

Idea: Based on the Bellman consistency equation, one can relate the value function to the immediate reward and the value function in the next state. Therefore, the labels can be generated online based on the current estimate of the values. Bootstrapping

Based on the Bellman consistency equation, we have

$$Q^{\pi}(s,a) = R(s,a) + \gamma \mathbb{E}_{\substack{s' \sim P(\cdot|s,a) \\ a' \sim \pi(\cdot|s')}} \left[Q^{\pi}(s',a') \right].$$

Therefore, upon observing

$$(s_t, a_t, r_t, s_{t+1}, a_{t+1}),$$

we can create a data point

$$\begin{cases} x^i = (s^i, a^i) = (s_t, a_t) \\ y^i = r_t + \gamma \hat{Q}^{\pi}(s_{t+1}, a_{t+1}) \end{cases}.$$

In this case, y^i will not be an unbiased estimator of Q^{π} in general:

$$\mathbb{E}_{\substack{s_{t+1} \sim P(\cdot|s_t,a_t) \\ A_{t+1} \sim \pi(\cdot|s_t)}} \left[y^i \mid s^i = s_t, a^i = a_t \right]$$

$$= \mathbb{E}_{\substack{s_{t+1} \sim P(\cdot|s_t,a_t) \\ A_{t+1} \sim \pi(\cdot|s_t)}} \left[R(s_t,a_t) + \gamma \hat{Q}^{\pi}(S_{t+1}, A_{t+1}) \right]$$

$$= R(s_t,a_t) + \gamma \mathbb{E}_{\substack{s_{t+1} \sim P(\cdot|s_t,a_t) \\ A_{t+1} \sim \pi(\cdot|s_t)}} \left[\hat{Q}^{\pi}(S_{t+1}, A_{t+1}) \right]$$

$$\neq \text{in general} \leftarrow \stackrel{?}{=} R(s_t,a_t) + \gamma \mathbb{E}_{\substack{s_{t+1} \sim P(\cdot|s_t,a_t) \\ A_{t+1} \sim \pi(\cdot|s_t)}} \left[Q^{\pi}(S_{t+1}, A_{t+1}) \right]$$

$$= Q^{\pi}(s_t,a_t)$$

* Nevertheless, y^i will have a lower variance compared to the MC-based method, in general.

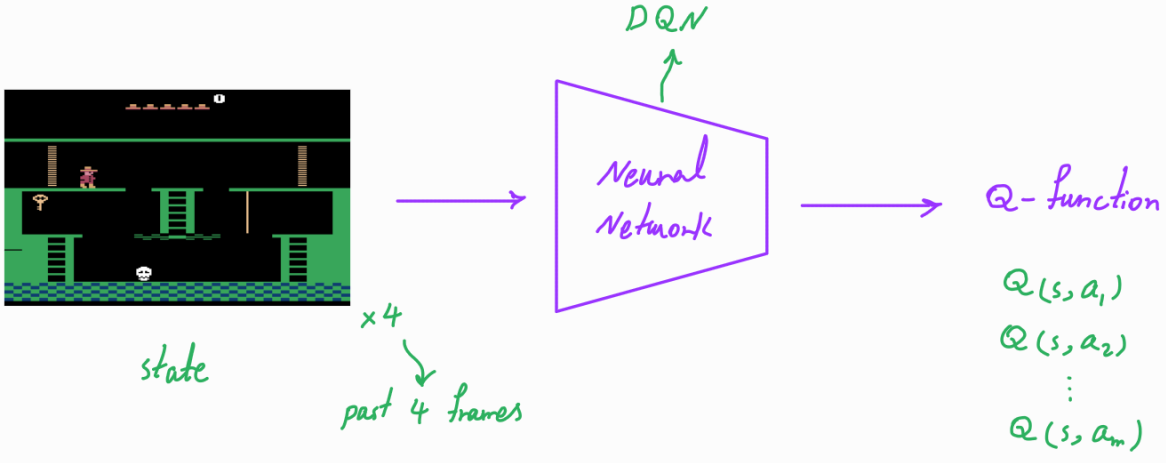
$\longrightarrow$

We can write the TD-learning-based counterparts of the PI-based algorithms with function approximation by adopting this data collection process.

On-policy method

Q-learning with function approximation: The Q-learning algorithm can be extended to case of large MDPs that require function approximation.

Neural network as function approximator $\longrightarrow$ Deep Q-learning



Success of DQN in playing Atari games

In Q-learning, the goal is to learn the optimal value function.

Based on the Bellman optimality equation, we have

$$Q^*(s,a) = R(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)} \left[\max_{a' \in \mathcal{A}} Q^*(s',a') \right].$$

Therefore, upon observing

$$(s_t, a_t, r_t, s_{t+1}),$$

we can create a data point

$$\begin{cases} x^i = (s^i, a^i) = (s_t, a_t) \\ y^i = r_t + \gamma \max_{a' \in \mathcal{A}} \hat{Q}(s_{t+1}, a') \end{cases}.$$

In this case, y^i will not be an unbiased estimator of Q^* in general:

$$\mathbb{E}_{s_{t+1} \sim P(\cdot|s_t,a_t)} \left[y^i \mid s^i = s_t, a^i = a_t \right]$$

$$= \mathbb{E}_{s_{t+1} \sim P(\cdot|s_t,a_t)} \left[R(s_t,a_t) + \gamma \max_{a' \in \mathcal{A}} \hat{Q}(s_{t+1}, a') \right]$$

$$= R(s_t,a_t) + \gamma \mathbb{E}_{s_{t+1} \sim P(\cdot|s_t,a_t)} \left[\max_{a' \in \mathcal{A}} \hat{Q}(s_{t+1}, a') \right]$$

$$\neq \text{in general} \leftarrow \stackrel{?}{=} R(s_t, a_t) + \gamma \, \mathbb{E}_{s_{t+1} \sim P(\cdot|s_t, a_t)} \left[\max_{a' \in \mathcal{A}} Q^*(s_{t+1}, a') \right]$$

$$= Q^*(s_t, a_t)$$

$\longrightarrow$

We can write a Q-learning algorithm with function approximation by adopting this data collection process and using TD-learning.

off-policy method

Collecting Labeled Data to Learn Q^*

Roll out the policy, the current policy or a soft policy (ε-greedy), to obtain the data points $(x^i = (s^i, a^i), y^i)$.

ε-greedy policy:

$$\pi(a|s) = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{|\mathcal{A}|} & \text{if } a = \arg\max_{a \in \mathcal{A}} \hat{Q}(s,a) \\ \frac{\varepsilon}{|\mathcal{A}|} & \text{otherwise} \end{cases}$$

Supervised Learning of Q^*

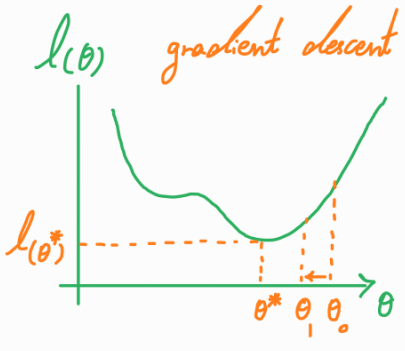
Based on the data collected, we can form an estimate $\hat{Q}$ by solving the following problem:

$$\hat{Q} = \underset{Q \in \mathcal{Q}}{\arg\min} \sum_{i=1}^{N} \left(Q(s^i, a^i) - y^i \right)^2 .$$

Let $\mathcal{Q}$ be a parametrized class of functions

$$\mathcal{Q} = \left\{ Q_\theta : \theta \in \mathbb{R}^K \right\}$$

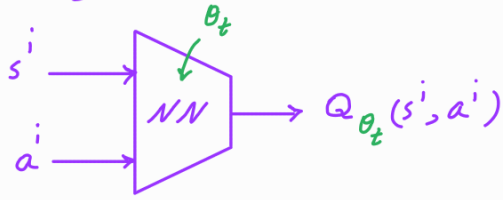
$\longrightarrow$ One can use gradient-based approaches to (approximately) solve this optimization problem.



$$\ell(\theta) = \sum_{i=1}^{N} \left(Q_\theta(s^i, a^i) - y^i \right)^2$$

$$\nabla_\theta \ell(\theta) = \sum_{i=1}^{N} 2 \left(Q_\theta(s^i, a^i) - y^i \right) \nabla_\theta Q_\theta(s^i, a^i)$$

backpropagation for a neural network



$\min_\theta \ell(\theta)$ $\longrightarrow$ gradient descent update rule:

$$\theta_{t+1} \longleftarrow \theta_t - \alpha_t \underbrace{\nabla_\theta \ell(\theta) \Big|_{\theta = \theta_t}}_{\nabla_\theta \ell(\theta_t)}$$

Note [Variants of Q-Learning]: Q-learning and Q-learning with function approximation, particularly DQN, have several variants that aim to accelerate or stabilize the learning process, such as

- Double DQN
  Decoupling selection and evaluation of actions
  to tackle overestimation bias of Q-learning

- Dueling DQN
  A network architecture separately representing
  the state value function and advantage function
  to improve generalization across actions

- Incorporating experience replay
  Sampling transitions from a replay buffer to remove correlations
  in the state sequence and smoothing the change in the data distribution

- Incorporating prioritized experiences
  Prioritizing sampling more informative (more recent,
  with higher TD error) transitions to improve data efficiency